



UNIVERSITÀ DI PARMA

Tipi di dato in C struct, union & enum

*Art and Science cannot exist but in minutely
organized particulars*

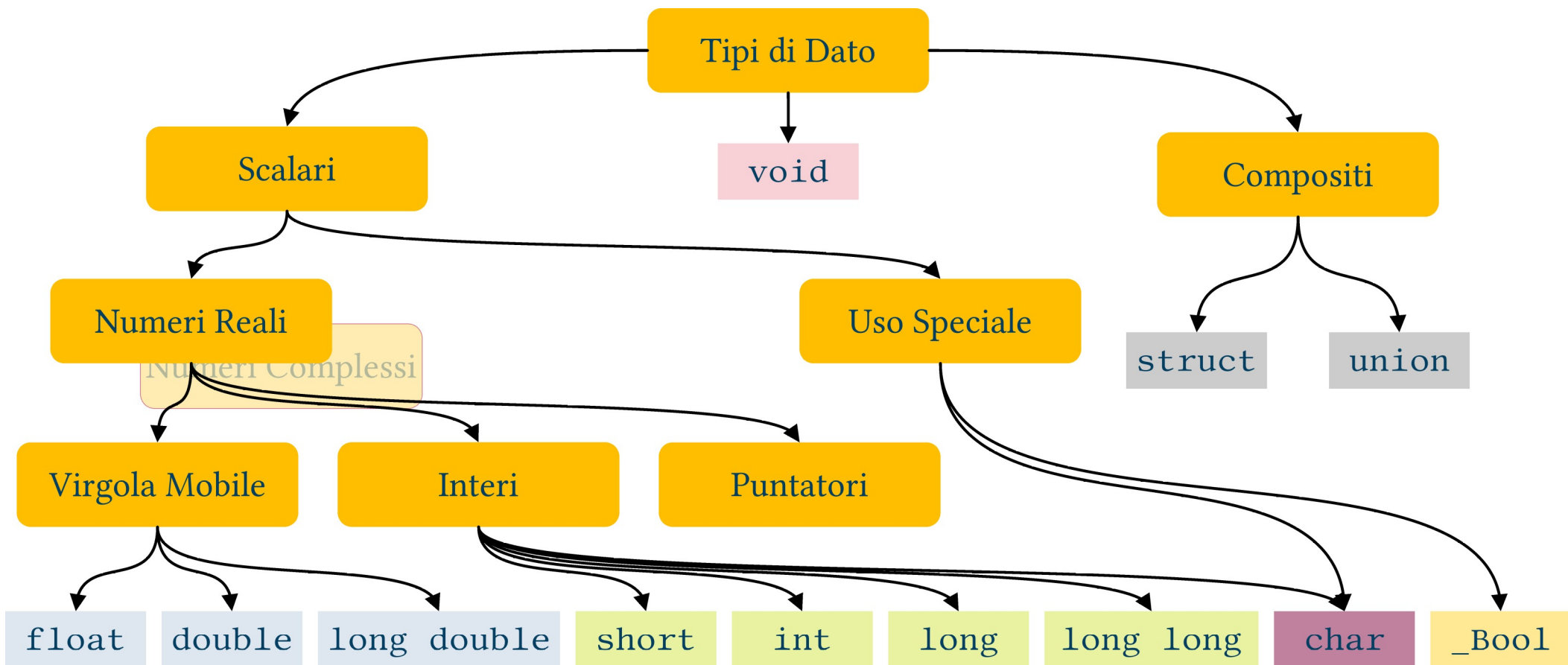
William Blake, To the Public

- Struct
- Union
- Enum

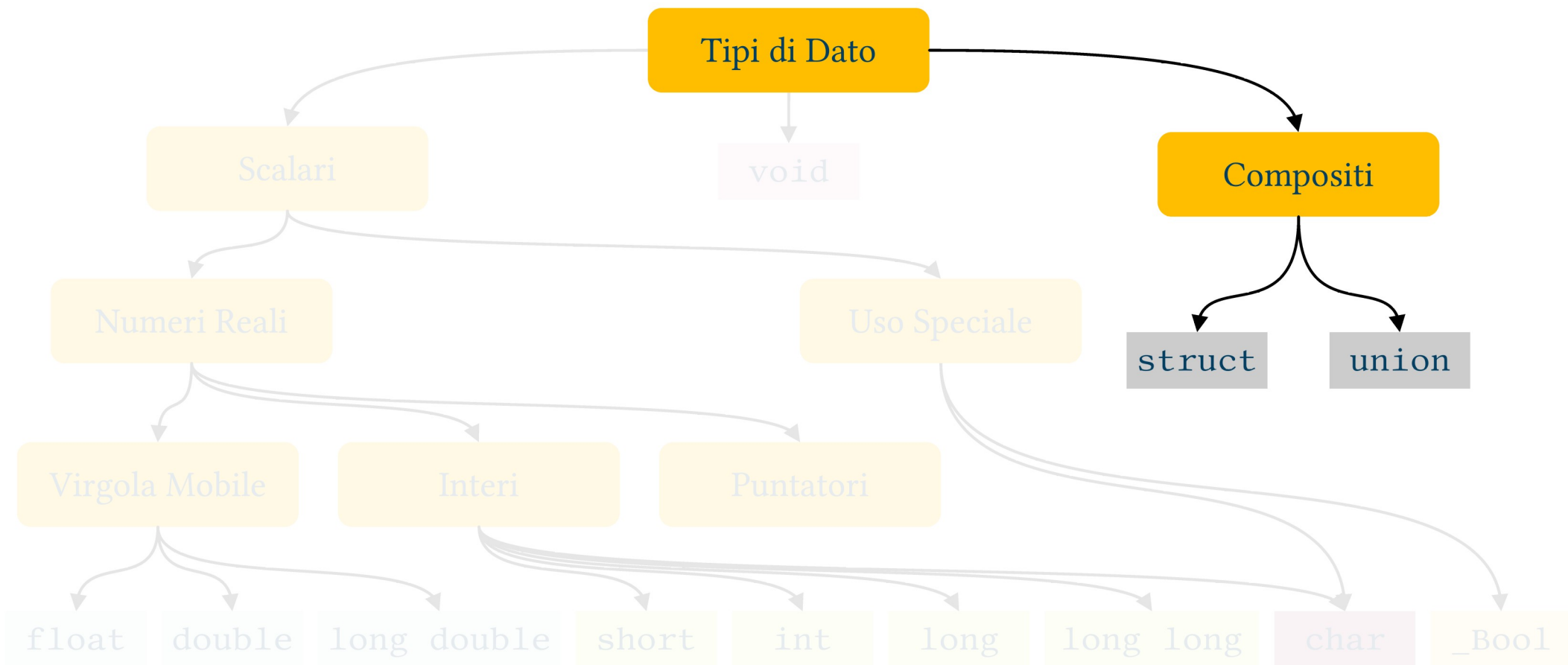
SUMMARY



C99 Albero dei Tipi di Dato



C99 Albero dei Tipi di Dato



Perché i dati compositi?

- Fino ad ora:
 - Numeri di differenti tipologie e intervalli
 - Caratteri e sequenze di caratteri
- Inefficiente per modellare mondo reale
 - Esempio: anagrafica studenti, mi servono:
 - Nome e cognome → stringhe
 - Matricola → numero intero
 - ...

- Il costrutto `struct` permette di
 - Aggregare dati anche di tipo differente
 - Definire nuove tipologie di dato
- Esempio:

```
struct studente{  
    char nome[100], cognome[100];  
    unsigned long matricola;  
};
```

- In questo esempio sto aggregando dati di tipo differente
- 2 array di char e un dato di tipo numerico intero
 - I singoli componenti si chiamano “elementi” o “membri”

```
struct studente{  
    char nome[100], cognome[100];  
    unsigned long matricola;  
};
```

- La “**struct studente**” è un nuovo tipo di dato
 - NON è una variabile
- Posso usarla per definire variabili

```
struct studente{  
    char nome[100], cognome[100];  
    unsigned long matricola;  
};
```

```
struct studente a, b, *c;
```


- Usually, the definition of `struct` constructs like “`struct student`” is placed at the beginning of the code just after `#include` directives
- Conversely the definition of variables can be placed where necessary
 - Inside functions, as function parameter... everywhere a variable can be defined

- Come accedo ai singoli elementi?
 - . per le variabili
 - -> per i puntatori

```
struct studente a, b, *c;
```

```
a.matricola = 12345;  
c->matricola = 67890;
```

- È possibile inizializzare le variabili di tipo “struct”?
- Approccio simile agli array
- Devo mantenere ordine elementi
- Inizializzazioni parziali $\rightarrow 0$

```
struct studente a={"Gino", "Pilotino", 12345};
```

- The “struct” keyword is, in fact, used to define a new data type
 - Like “int”, “float”, “char”...
- Therefore also arrays of structs are possible
 - Elements are accessed using the usual notation ([]) just before the dot . operator

```
struct studente a, b, *c, d[100];
```

```
c[12].matricola = 67890;
```

- Le struct possono contenere altre struct

```
struct anagrafica{  
    char nome [100], cognome[100];  
    int giorno, anno, mese;  
};
```

```
struct studente{  
    struct anagrafica dati;  
    char matricola[12];  
};
```

- Particolare importanza rivestono le struct che contengono puntatori allo stesso tipo di dato
 - Liste, alberi, grafi ecc.
 - Non le vedremo

```
struct nodo{  
    char nome [100], cognome[100];  
    struct nodo *successivo;  
};
```

- Structs are also used to “pack” data exploiting “Bit Fields”
- Namely, we can set the number of bits for each struct member
- Use the same unsigned type for each field
- Reason:
 - Efficient memory use
 - I/O with binary files

- Usually unsigned types are used
- Syntax:

```
struct printer {  
    unsigned int status : 2;  
    unsigned int data    : 8;  
    unsigned int paper   : 2;  
    unsigned int error   : 4;  
};
```


- Stessa sintassi struct
- Ma: elementi tutti sovrapposti in memoria
 - Sistemi embedded
 - Gestione HW
 - Flessibilità

```
union nodo{  
    int    intero;  
    double floating;  
} mixed_array[100];
```

- Costrutto per definire tipicamente elenchi di valori costanti
 - Di fatto di tipo “int”
- Spesso usato come argomento funzioni

```
enum stagioni{primavera, estate, autunno, inverno};
```

```
enum stagioni a;
```

```
a = primavera;
```



UNIVERSITÀ DI PARMA

Tipi di dato in C struct, union & enum



KEEP
CALM
IT'S
QUESTION
TIME

*Art and Science cannot exist but in minutely
organized particulars*

William Blake, To the Public